



**WYDZIAŁ
ELEKTROTECHNIKI
I INFORMATYKI**
POLITECHNIKI RZESZOWSKIEJ

Aplikacja bankowa

Mikołaj Hus – 179503

Rzeszów, 15 czerwca 2026

Spis treści

1. Wstęp i Cel Projektu	2
2. Architektura Systemu	3
2.1. Separacja Warstw i Komponenty	3
3. Model Bazy Danych	4
3.1. Diagram ERD	4
3.2. Tabela Użytkowników (users)	4
3.3. Tabela Kont Bankowych (accounts)	4
3.4. Tabela Kont Bankowych (transactions)	5
3.5. Tabela Kont Bankowych (activity_logs)	5
4. Kluczowe Mechanizmy i Algorytmy	6
4.1. Transakcyjność Operacji Bankowych	6
5. Bezpieczeństwo Systemu	7
6. Podsumowanie i Wnioski	8

1. Wstęp i Cel Projektu

Celem projektu było zaprojektowanie i implementacja desktopowej aplikacji bankowej. System został opracowany przy użyciu języka C++ oraz frameworka Qt (wersja 6), natomiast warstwa przechowywania danych opiera się na relacyjnej bazie danych PostgreSQL.

Aplikacja zawiera następujące funkcje:

- poziom użytkownika:
 - dane szczegółowe konta (saldo, numer, itp.)
 - podział na podkonta (konto główne, lokata)
 - historia transakcji
 - możliwość dokonywania przelewów
- poziom administratora:
 - zarządzanie użytkownikami (np. blokowanie)
 - logi aktywności użytkowników
 - system uprawnień

2. Architektura Systemu

Projekt opiera się na architekturze modułowej.

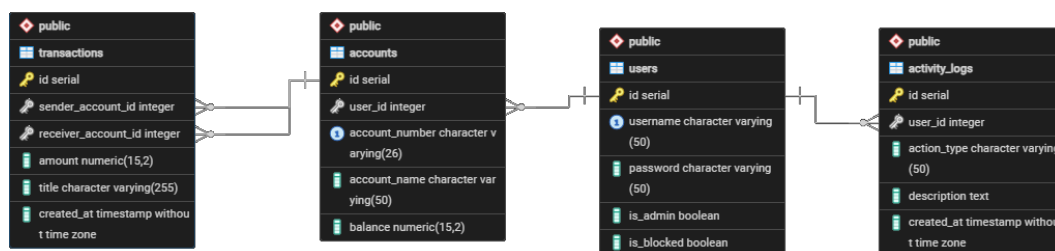
2.1. Separacja Warstw i Komponenty

- **DatabaseManager:** Statyczna klasa centralna odpowiedzialna za inicjalizację połączenia z bazą danych poprzez sterownik QPSQL, bezpieczne zamykanie sesji oraz globalny zapis logów.
- **BankApp (Główne Okno):** Pełni rolę głównego kontrolera aplikacji. Nasłuchuje sygnałów z podmodułów i realizuje funkcje pulpitu klienta.
- **LoginWidget / RegisterWidget:** Niezależne moduły interfejsu użytkownika realizujące odpowiednio autoryzację oraz rejestrację nowych kont.
- **TransferDialog / HistoryDialog / AdminWidget:** Autonomiczne okna dialogowe wywoływane w kontekście konkretnych akcji.

3. Model Bazy Danych

Baza danych została znormalizowana i zaprojektowana w systemie PostgreSQL. Poniżej przedstawiono strukturę tabel relacyjnych.

3.1. Diagram ERD



3.2. Tabela Użytkowników (users)

Przechowuje dane uwierzytelniające oraz flagi uprawnień i blokad systemu:

- `id` (SERIAL, PRIMARY KEY) – Unikalny identyfikator użytkownika.
- `username` (VARCHAR, UNIQUE) – Login klienta.
- `password` (VARCHAR) – Hasło klienta.
- `is_admin` (BOOLEAN) – Definiuje uprawnienia administracyjne.
- `is_blocked` (BOOLEAN) – Określa blokadę dostępu do systemu.

3.3. Tabela Kont Bankowych (accounts)

Przechowuje dane dotyczące kont bankowych:

- `id` (SERIAL, PRIMARY KEY) – Unikalny identyfikator konta bankowego.
- `user_id` (INTEGER, FOREIGN KEY) – Identyfikator klienta do którego należy konto.
- `account_number` (VARCHAR) – Numer konta bankowego.
- `account_name` (VARCHAR) – Nazwa konta bankowego.
- `balance` (NUMERIC) – Ilość środków na koncie.

3.4. Tabela Kont Bankowych (**transactions**)

Przechowuje dane dotyczące przelewów:

- **id** (SERIAL, PRIMARY KEY) – Unikalny identyfikator transakcji.
- **sender_account_id** (INTEGER, FOREIGN KEY) – Identyfikator konta źródłowego przelewu.
- **receiver_account_id** (INTEGER, FOREIGN KEY) – Identyfikator konta docelowego przelewu.
- **amount** (NUMERIC) – Ilość środków przesłanych w przelewie.
- **title** (VARCHAR) – Nazwa przelewu.
- **created_at** (TIMESTAMP) – Data wykonania przelewu.

3.5. Tabela Kont Bankowych (**activity_logs**)

Rejestruje zdarzenia systemowe (np. LOGIN_FAILED, ADMIN_ACTION) w celach bezpieczeństwa.

- **id** (SERIAL, PRIMARY KEY) – Unikalny identyfikator transakcji.
- **sender_account_id** (INTEGER, FOREIGN KEY) – Identyfikator konta źródłowego przelewu.
- **receiver_account_id** (INTEGER, FOREIGN KEY) – Identyfikator konta docelowego przelewu.
- **amount** (NUMERIC) – Ilość środków przesłanych w przelewie.
- **title** (VARCHAR) – Nazwa przelewu.
- **created_at** (TIMESTAMP) – Data wykonania przelewu.

4. Kluczowe Mechanizmy i Algorytmy

4.1. Transakcyjność Operacji Bankowych

Wszystkie operacje modyfikujące stanu konta realizowane są wewnątrz transakcji bazodanowej w celu zapewnienia integralności danych. Przepływ algorytmu przedstawia się następująco:

```
QSqlDatabase db = QSqlDatabase::database();
if (db.transaction())
{
    // 1. Sprawdzenie odbiorcy i pobranie ID konta
    // 2. Aktualizacja konta nadawcy (UPDATE accounts SET balance =
balance - X)
    // 3. Aktualizacja konta odbiorcy (UPDATE accounts SET balance =
balance + X)
    // 4. Logowanie w tabeli transactions

    if (wszystkie_operacje_ok)
    {
        db.commit(); // Dane zostają trwale zapisane
    }
    else
    {
        db.rollback(); // Stan bazy wraca do punktu wyjścia
    }
}
```

5. Bezpieczeństwo Systemu

Wdrożony system realizuje podstawowe standardy bezpieczeństwa aplikacji teleinformatycznych:

- 1) **Ochrona przed SQL Injection:** Wszystkie zapytania dynamiczne konstruowane są przy użyciu mechanizmu *Prepared Statements* (metoda `query.prepare()`), co uniemożliwia wstrzyknięcie złośliwego kodu SQL.
- 2) **Zarządzanie sesją:** Dane wrażliwe użytkownika (takie jak unikalny identyfikator sesji `loggedUserId`) przechowywane są wyłącznie w prywatnej pamięci operacyjnej aplikacji i są czyszczone w momencie wywołania procedury wylogowania.
- 3) **Audytowalność (Non-repudiation):** Każda operacja administracyjna oraz anomalie (np. próba logowania na zablokowane konto) generują trwały ślad w tabeli `activity_logs`.

6. Podsumowanie i Wnioski

Zaimplementowana aplikacja stanowi stabilną podstawę dla systemów typu e-banking. Zastosowanie modularnej struktury w C++ ułatwia ewentualną rozbudowę aplikacji o nowe funkcjonalności, takie jak generowanie wyciągów do plików CSV czy implementacja mechanizmu lokat terminowych. Silnik PostgreSQL gwarantuje bezpieczeństwo i spójność danych nawet w przypadku niespodziewanych awarii zasilania po stronie klienta.